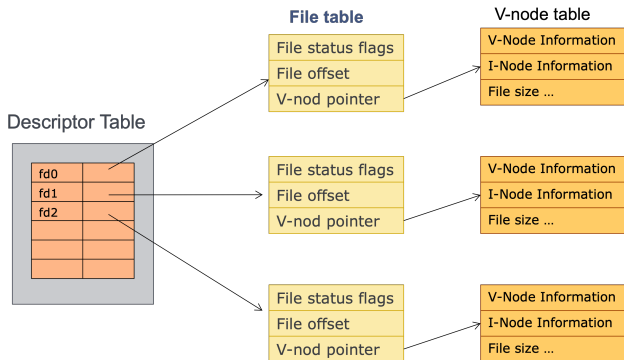


File Sharing

File Sharing

- Linux or Unix supports the sharing of open file among different processes.
- Kernel use three data structure for managing processes interacting with files.
 - A **descriptor table** (one table per process) that stores
 - file descriptors
 - pointers to file table (s)
 - A **File table**(one for each open file) that stores
 - File status flags (read, write, append, ...)
 - The current file offset
 - A pointer to the virtual-node (aka. v-node) table for the file
 - A **v-node table** for each open file stores
 - Information on file type and file size
 - Pointer to the functions that operate on the file
 - i-node information

An Example Relationship Among Descriptor, Open File, and V-Node Tables

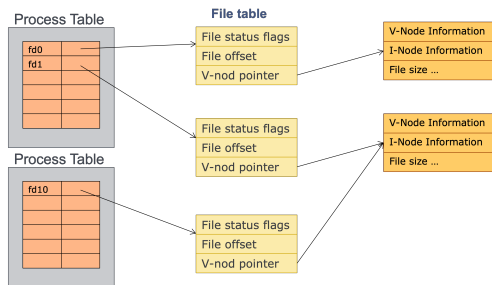


Note: The descriptor table can also be referred to as process table.

File Sharing Scenarios

How kernel handle scenarios when two processes need to share a file?

- One process write to, another process read from a file
- Both process append to a file
- Both processes need to do lseek()
Since each process has file table, based on the file status flag, before read, write, or lseek, check file size which is saved in v-table

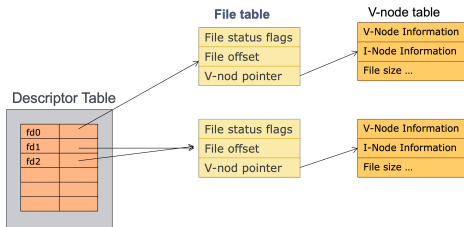


File Sharing - dup() System Call

The `dup()` system call creates a copy of a file descriptor.

- The new file descriptor returned by `dup()` is guaranteed to be the lowest-numbered available file descriptor.
- If the copy is successfully created, then the original and copy file descriptors may be used interchangeably.
- They both refer to the same open file and thus share file offset and file status flags.

```
#include <unistd.h>
// returns a new file descriptor if success, -1 on error.
int dup(int fd);
```



dup() System Call Example

```
#include <unistd.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>

#define FILE_MODE (S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH)

char buf[] = "I'm using dup! ";
char buf1[] = "I'm also using dup!!";

void err_sys(char *str){
    printf ("%s",str);
    exit (1);
}

int main(){
    int fd, fd1;

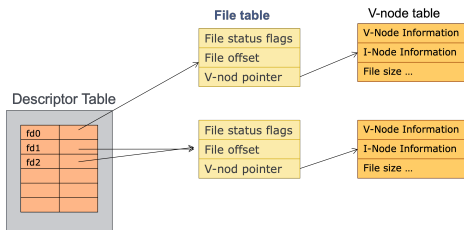
    if ((fd = creat("out.txt", FILE_MODE)) < 0)
        err_sys("Creat File Error");
    if (write (fd, buf, sizeof(buf)) != sizeof(buf))
        err_sys("Write Error");
    if ((fd1= dup(fd)) < 0)
        err_sys("Creat Descriptor duplication error");
    if (lseek (fd1, sizeof(buf)+1, SEEK_SET) == -1)
        err_sys("Creat seekl Error"); /*now offset =40 */
    if (write (fd1, buf1, sizeof(buf1)) != sizeof(buf1))
        err_sys("Creat Write Error");
    close(fd);
    return 0;
}
```

File Sharing - dup2() System Call

The `dup2()` system call is similar to `dup()` but the basic difference between them is that instead of using the lowest-numbered unused file descriptor, it uses the descriptor number specified by the user.

- If the descriptor `newfd` was previously open, it is silently closed before being reused.
- If `oldfd` is not a valid file descriptor, then the call fails, and `newfd` is not closed.
- If `oldfd` is a valid file descriptor, and `newfd` has the same value as `oldfd`, then `dup2()` does nothing (i.e. without closing it), and returns `newfd`.

```
#include <unistd.h>
// returns newfd on success, -1 on error.
int dup2(int oldfd, int newfd);
```



dup2() System Call Example

```
/* output redirection with dup2() system call. */
#include <stdlib.h>
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
void err_sys(char *);

int main(int argc, char **argv){
    int pid;
    int newfd;      /* new file descriptor */
    if (argc != 2)
        err_sys("usage: %s output file name\n");
    if ((newfd = open(argv[1], O_CREAT|O_TRUNC|O_WRONLY, 0644)) < 0)
        err_sys("usage: open file error.\n");
    printf("This goes to the standard output.\n");
    printf("Now the standard output will go to \"%s\".\n", argv[1]);

    /* standard output is file descriptor 1, so we use dup2
       to copy the new file descriptor onto file descriptor 1
       dup2 will close the current standard output after dup2,
       standard output is closed and new file will become the standard output */
    dup2(newfd, 1);
    printf("Now this goes to the new file, since new file becomes std output\n");
    exit(0);
}

void err_sys(char *str){
    printf ("%s",str);
    exit (1);
}
```


In-depth dup2() System Call Example

```
#include <stdlib.h>
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
void err_sys(char *);

int main(int argc, char **argv){
    int pid;
    int newfd;      /* new file descriptor */
    char *cmd[] = { "/bin/ls", "-la", "/", 0 };
    if (argc != 2)
        err_sys("usage: %s output file name\n");
    if ((newfd = open(argv[1], O_CREAT|O_TRUNC|O_WRONLY, 0644)) < 0)
        err_sys("usage: open file error.\n");
    printf("writing output of the command %s to \"%s\"\n", cmd[0], argv[1]);
    /* This new file will become the standard output after dup2.*/
    dup2(newfd, 1);
    /* now we run the command. It runs in this process and will have */
    /* this process' standard input, output, and error */
    if ((execvp(cmd[0], cmd))<0)
        err_sys("usage: execvp failed.\n");
    return 0;
}

void err_sys(char *str){
    printf ("%s",str);
    exit (1);
}
```

Real Application Scenarios of File Sharing

The file descriptor is duplicated (by default) in the child processes to share file table entry.

```
#/bin/sh
# export.sh: export variables to child scripts
export a="Cool Cool"
export b="Fun Fun"
./import1.sh
./import2.sh
exit 0
```

```
#!/bin/sh
# import1.sh
echo "$a"
exit 0
```

```
#!/bin/sh
# import2.sh
echo "$b"
exit 0
```

```
./export.sh
```

Quiz

Can you think of another application using file sharing?